基于 FPGA 图形加速的游戏掌机

一、摘要

在嵌入式人机交互及 2D 游戏等应用中，图像系统需要在固定的刷新周期内完成背景绘制、动态图像更新和多图层透明叠加。采用 CPU 进行纯软件渲染，逐像素操作与频繁访存会占用大量处理时间，随着同屏元素的大量增加，易出现帧率下降的现象，并挤占其他应用逻辑的执行时间。

本作品基于易灵思 Ti60F225 FPGA，设计了一款采用硬件图形加速的游戏掌机。系统由 RISC-V 软核负责游戏逻辑、按键响应及渲染指令下发，由专用硬件渲染器完成像素搬运与透明混合。针对多款二维游戏中的大量对象绘制及多图层叠加需求，通过批量指令、坐标生成弹幕和片上条带缓存减少指令提交与重复访存，在有限的片上资源与存储带宽下提高图形处理能力。本文解决的问题主要体现在以下几个方面：

① 针对 CPU 逐像素绘制效率低的问题，设计指令驱动的硬件渲染器，支持矩形填充、图像搬运、透明色剔除、水平镜像及 Alpha 混合，将重复的像素处理交由硬件执行。

② 针对大量对象逐个提交命令的开销，设计批量精灵与弹幕指令，通过坐标列表组织同类对象绘制；对规则圆形弹幕按坐标生成颜色，并用片上覆盖缓存合并同一批次的重叠像素，减少指令提交、素材读取与重复写入。

③ 针对 Alpha 混合反复读写背景像素的问题，设计片上条带缓存，在缓存内完成同一区域的多精灵叠加后集中写回，结合素材缓存、多像素并行计算与突发传输，降低 DDR 访问开销。

本系统实现了 640×480 分辨率、标称 60Hz 的图像输出，集成东方红魔乡、中国象棋、扫雷、2048、贪吃蛇、地牢幸存者、推箱子和俄罗斯方块八款游戏，通过统一图标首页完成选择与切换。系统提供 CPU 与 FPGA 同屏对比，左右两侧初始各绘制 10000 颗 8×8 弹幕，分别显示实际更新帧率和绘制耗时；另设 8×8、16×16、32×32 三种硬件弹幕测试，初始数量分别为 40000、20000、7000 颗。Alpha 与透明色叠加测试初始使用 5000 个 30×30 双姿态精灵，数量与透明度可独立调节，用于测量不同条件下的渲染能力。

关键词：FPGA；RISC-V；二维图形渲染；批量指令；条带缓存；Alpha 混合；游戏掌机

二、作品概述

2.1 功能与特性

本作品是一套基于易灵思 FPGA 的 2D 图像渲染加速引擎，由 RISC-V 软核下发绘制指令，专用硬件完成像素处理，并通过 HDMI 输出 640×480 分辨率、约 60Hz 的图像。主要功能与特性如下：

① 支持矩形填充与图像搬运，可完成画面清除、背景绘制及动态图像更新。

② 支持透明色剔除、水平镜像与可调 Alpha 混合，实现不规则图像绘制、左右翻转及多图层透明叠加。

③ 支持批量精灵与弹幕绘制，一条命令可描述多个同类对象，降低大量元素绘制时的指令提交开销。

④ 采用片上条带缓存、素材缓存与多像素并行处理，提高多精灵 Alpha 混合效率，减少外部存储访问。

⑤ 采用双帧缓存与帧同步切换，避免画面撕裂，并提供访存范围保护和异常诊断机制。

⑥ 提供 CPU 与 FPGA 同屏渲染对比、三种尺寸的硬件弹幕测试及透明混合测试，实时显示绘制耗时与实际更新帧率，便于在相同条件下评估渲染性能。

⑦ 提供八游戏统一入口，采用图标网格组织应用，支持方向键选择、确认进入及返回上级界面；各游戏共用渲染驱动、素材管理和双帧缓存机制。

2.2 应用领域

本作品主要面向掌上游戏与嵌入式互动娱乐，通过统一入口运行射击、棋类、益智及休闲游戏，为不同类型的二维游戏提供按键输入、素材管理和图形绘制支持。其中，弹幕射击和地牢幸存者用于展示大量动态对象及透明特效，棋类与益智游戏则体现统一平台对不同交互方式的适应能力。系统中的硬件渲染器与应用逻辑相互分离，除游戏外，也可扩展用于嵌入式操作面板、动态图标和仪表界面等图形显示场景。

2.3 主要技术特点

系统采用 RISC-V 软核与专用渲染器协同架构，通过寄存器接口和异步命令 FIFO 传递绘制任务，实现应用逻辑与像素处理的分工。渲染器通过 AXI 总线访问 DDR3，结合突发传输与多像素并行处理提高数据吞吐率。针对大量同类对象，采用坐标列表描述绘制位置，以批量指令减少逐个提交的开销；针对规则圆形弹幕，根据像素坐标直接生成颜色，省去贴图读取，并在片上覆盖缓存中合并同一批次的重叠像素，集中写回 DDR。针对多精灵 Alpha 混合，采用片上条带缓存与素材缓存，在片上完成局部区域的连续叠加后集中写回，减少重复访存。显示端采用双帧缓存与帧同步切换，并通过跨时钟域同步、访存范围保护和异常状态监测保障数据传输与画面输出的可靠性。

2.4 主要性能指标

表 2.1 展示最终工程的资源占用与时序裕量，数据以布局布线报告为准。

表 2.2 列出定型版本的测试项目及条件，测试结果为上板实测。模式的初始对象数量不等同于保持刷新率时的数量极限。

表 2.1 工程占用的资源量

XLRS	Memory Blocks	DSP Blocks	WNS	WHS
42695/60800	192/256	36/160	0.059ns	0.023ns

表 2.2 性能测试项目、条件及结果

测试项目	测试条件	测试结果
视频输出	640×480 HDMI 输出，RGB565 帧缓存，标称 60Hz。	实测刷新率：59.52 Hz
CPU 与 FPGA 同屏对比	左侧 FPGA、右侧 CPU，初始各 10000 颗 8×8 三色圆球；相同绘制区域尺寸与运动规则，数量按 500 颗调节。	FPGA：59.52 FPS 绘制耗时 10.11 ms CPU：9.92 FPS 绘制耗时 66 ms
GPU 8×8 弹幕测试	黑色背景，初始 40000 颗；数量按 500 颗调节，范围 500～48000 颗。	59.32 FPS 绘制耗时 10.55 ms 稳定上限：40000 颗
GPU 16×16 弹幕测试	黑色背景，初始 20000 颗；数量按 500 颗调节，范围 500～48000 颗。	59.32 FPS 绘制耗时 13.20 ms 稳定上限：20000 颗
GPU 32×32 弹幕测试	黑色背景，初始 7000 颗；数量按 500 颗调节，范围 500～48000 颗。	59.32 FPS 绘制耗时 14.52 ms 稳定上限：7000 颗
Alpha 与透明色叠加测试	初始 5000 个 30×30 双姿态精灵，Alpha=128/256；Color Key 开启，屏内反弹；数量与 Alpha 独立调节。	数量：5000 只 Alpha：128/256 59.52 FPS
上电独立运行	完整 8MiB 镜像包含 PL、压缩 PS 和八游戏素材；断电约 5 秒后重新上电，检查首页及游戏入口。	启动结果：正常 功能检查：完整

2.5 主要创新点

① 坐标列表驱动的批量绘制。将同类对象的素材、尺寸及绘制属性统一配置，通过坐标列表描述各对象的位置，由硬件连续完成绘制。相比逐个提交命令，减少重复参数传输与寄存器访问，降低大量精灵、弹幕场景下的软核提交开销。

② 基于坐标生成与覆盖缓存的弹幕绘制。圆球按局部坐标生成彩色外环、白色圆心和透明外角，省去贴图读取。同批次的不透明圆球先在片上保存最终覆盖结果，再按行突发写回，减少重叠像素的重复写入，支持 8×8、16×16 和 32×32 三种尺寸。

③ 面向多精灵混合的片上条带缓存。按条带划分目标画面，将背景像素读入片上缓存，按绘制顺序完成与该区域相交的精灵叠加，再集中写回 DDR。结合素材缓存与多像素并行计算，减少重叠区域反复读写，提高 Alpha 混合与透明色剔除同时启用时的渲染效率。

2.6 设计流程

整个系统的设计流程如图 2.1 所示



图 2.1 系统设计流程

① 以官方例程为基础，配置时钟、复位、管脚及外设接口，分别验证 RISC-V 软核运行、串口通信、DDR 读写与 HDMI 输出，确保基础硬件通路正常工作。

② 设计 APB 命令寄存器接口、异步命令 FIFO 与 AXI 读写逻辑，实现矩形填充、双帧缓存及帧同步切换，通过移动色块验证指令下发、硬件绘制和显示输出的完整通路。

③ 整理游戏素材，将图像转换为 RGB565 格式，统一存储方式与 Flash 地址分配。编写素材初始化程序，将 Flash 中的图像搬入 DDR，并通过数据校验确认搬运结果。

④ 在基础填充功能上加入图像搬运、透明色剔除、水平镜像及 Alpha 混合。针对不同尺寸、地址对齐、图像边界及总线背压等情况编写仿真用例，并通过上板显示验证渲染结果。

⑤ 根据板卡接口与电平要求，设计独立键盘电路，完成电路板焊接和测试。在 FPGA 内实现输入同步与独立消抖，通过寄存器向软核提供按键状态，实现角色移动、攻击及菜单控制。

⑥ 开发角色、敌人、弹幕、碰撞检测和关卡切换等游戏逻辑，并加入纯 CPU 与硬件渲染对比及 Alpha 混合压力测试。通过分段计时定位瓶颈，逐步进行突发传输、批量指令、坐标生成弹幕、片上条带缓存及弹幕覆盖缓存等优化。

⑦ 完善访存范围保护、开展连续运行和高负载测试。确认功能鲁棒性后，将硬件配置、软核程序与素材合并为完整固化文件，验证上电独立运行。

三、系统组成及功能说明

3.1 整体介绍

系统整体框图如图 3.1 所示

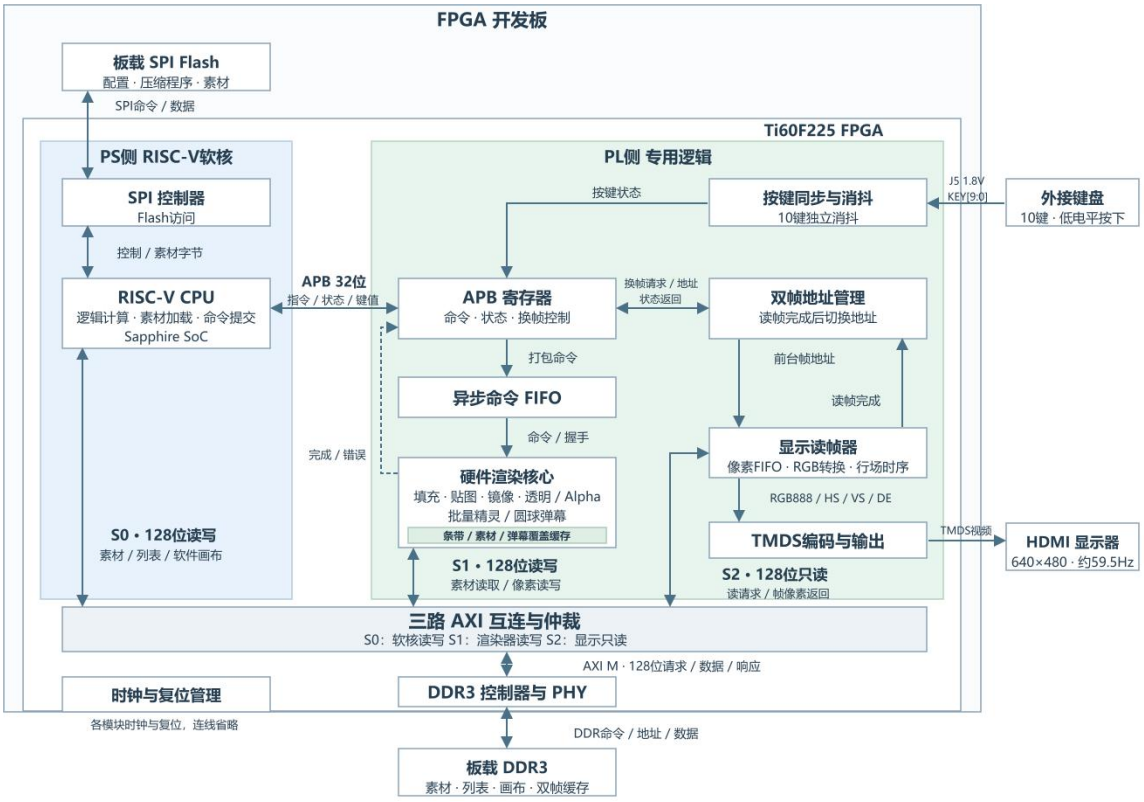


图 3.1 系统整体框图

① 系统总体组成

（1）FPGA 开发板以易灵思 Ti60F225 为核心，内部按功能划分为 RISC-V 软核侧（PS 侧）与专用逻辑侧（PL 侧）。PS 侧负责应用逻辑与绘制任务组织，PL 侧负责命令执行、像素处理及显示输出，两部分均在 FPGA 内部实现。

（2）板载 SPI Flash 保存硬件配置、软核程序和图像素材；DDR3 保存运行期素材、坐标列表、场景描述表及两组帧缓存。外接键盘提供操作输入，HDMI 显示器接收最终画面。

② PS 侧处理与素材初始化

（1）PS 侧采用 Sapphire RISC-V 软核系统，负责菜单控制、游戏逻辑、对象运动、碰撞判定及性能统计，并根据当前场景组织绘制任务。

（2）素材初始化时，CPU 通过 SPI 控制器读取 Flash 中的图像数据，再经 AXI S0 写入 DDR3 素材区，完成校验后供硬件渲染器使用。该过程由 CPU 执行，不经过按键模块或渲染命令 FIFO。

（3）运行时，CPU 将同类对象的坐标或场景描述写入 DDR3，通过 APB 提交素材地址、目标地址、尺寸及绘制属性。同屏对比中的 CPU 侧自行完成软件画布的清屏、坐标计算与逐像素绘制；画布完成后，由硬件复制到显示帧的 CPU 区域。该复制只呈现已完成的结果，不参与 CPU 侧弹幕像素生成。

③ 按键输入与 PS、PL 交互

（1）外接键盘通过 J5 向 FPGA 输入十路按键信号，按下时为低电平。按键模块对输入进行同步和独立消抖，将稳定的按键状态送入 APB 寄存器，CPU 通过读取寄存器响应操作。

（2）APB 寄存器是软核与专用逻辑之间的控制接口，接收绘制参数、换帧请求和通路复位控制，同时提供按键、FIFO、渲染完成及异常状态的回读。

（3）绘制参数在提交时打包为命令，写入异步命令 FIFO，再由渲染核心取出执行。FIFO 隔离外设与渲染时钟域，并缓冲短时提交峰值；渲染完成与错误信息经跨时钟域处理后返回 APB 寄存器，供 CPU 判断任务状态。

④ PL 侧硬件渲染

（1）render_core 解析 FIFO 中的命令，完成矩形填充、图像搬运、水平镜像、透明色剔除和 Alpha 混合。对于批量绘制命令，渲染核心从 DDR3 读取坐标或场景描述表，连续处理多个对象。

（2）规则圆形弹幕根据局部像素坐标生成彩色外环与白色圆心。满足条件的批次先在片上覆盖缓存中合并重叠像素，再集中写回。普通图像则从 DDR3 素材区读取像素，按命令指定的位置和属性写入目标帧缓存。

（3）Alpha 条带缓存、素材缓存和弹幕覆盖缓存均位于渲染核心内部。条带模式将目标区域读入片上，按顺序叠加精灵后集中写回；覆盖缓存只处理不透明规则圆球，记录同批次的最终覆盖结果。两条通路由渲染核心共用 AXI S1，不增加 DDR 总线接口。

⑤ 三路 AXI 与共享存储

（1）AXI S0 连接 RISC-V 软核，承担素材加载、列表更新及纯 CPU 绘制等读写任务；AXI S1 连接渲染核心，承担素材、描述表及目标像素的读写；AXI S2 连接显示读帧器，仅使用读通道读取前台帧缓存。三组接口的数据宽度均为 128 位。

（2）三路请求进入 AXI 互连与仲裁模块，汇合后通过 M 端口访问 DDR3 控制器。控制器与 PHY 将 AXI 访问转换为 DDR 命令及数据传输，并将读取数据和写入响应送回对应请求模块。

（3）DDR3 中的素材、列表和帧缓存按地址分区管理。正常绘制时，CPU 或渲染器更新后台帧缓存，显示读帧器读取前台帧缓存，通过分离读写对象避免显示尚未完成的画面。

⑥ 帧缓存切换与显示输出

（1）后台帧绘制完成后，CPU 通过 APB 提交下一帧地址及换帧请求。双帧地址管理器保存该请求，等待显示读帧器完成当前帧的 DDR 读取后，更新前台帧地址，并向 CPU 返回换帧状态。

（2）显示读帧器通过 AXI S2 读取前台帧数据，写入异步像素 FIFO，再在像素时钟域取出 RGB565 像素。模块同时完成 RGB565 至 RGB888 的转换，并产生行同步、场同步及数据有效信号。

（3）RGB888 像素及 HS、VS、DE 信号送入 TMDS 编码模块，经 FPGA I/O 串行输出至 HDMI 显示器，形成 640×480、约 59.5Hz 的连续画面。

3.2 硬件系统介绍

① 硬件整体介绍

整个硬件系统的实物图如图 3.2 所示，主控为图片右上方易灵思的 Ti60F225 开发板，左上方为使用到的 HDMI 显示屏，下方为掌机的供电系统与按键系统的 PCB。

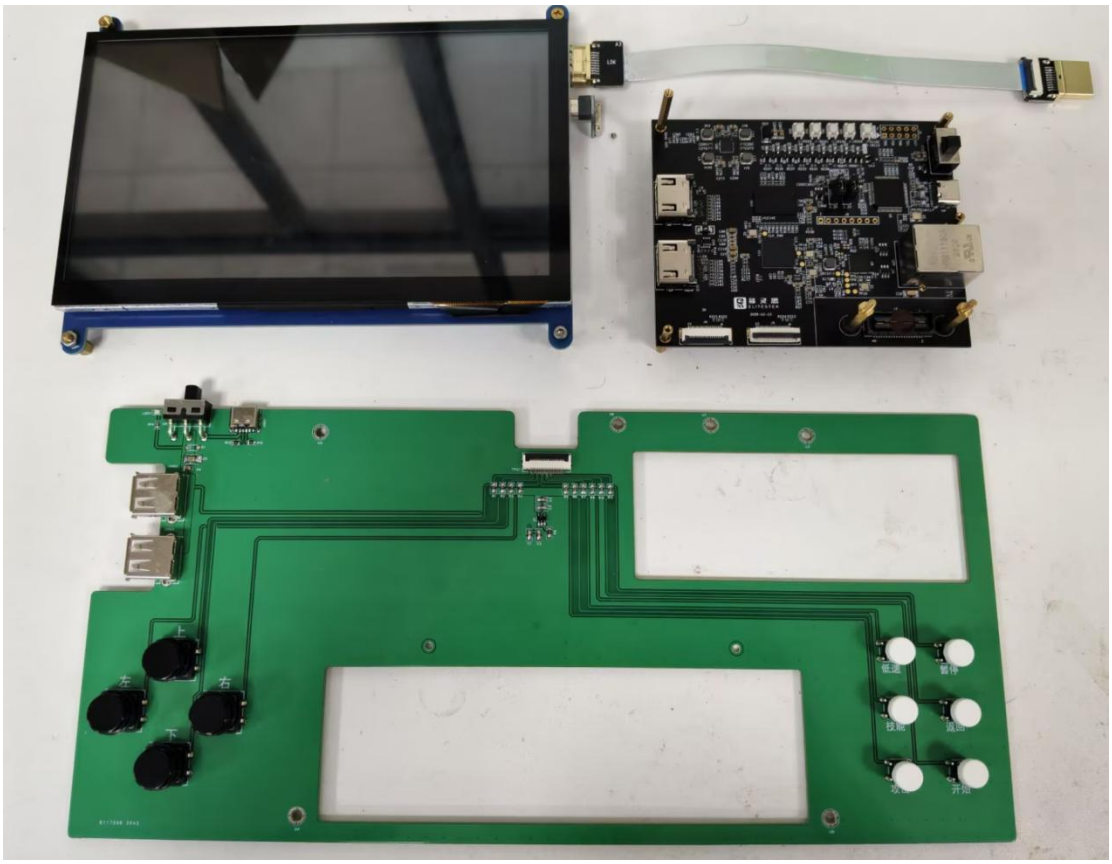


图 3.2 硬件系统实物图

② 电路模块介绍

本文使用的外部电路模块只有掌机的 PCB，故只介绍该 PCB 的电路结构。

（1）键盘对外接口

自制键盘的对外接口如图 3.3 所示，有十个按键电平输出

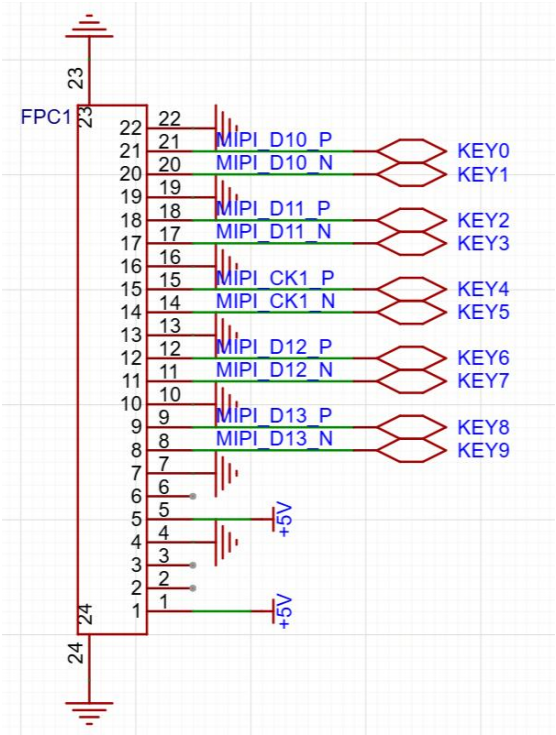


图 3.3 键盘对外接口原理图

（2）底板电源电路

底板电源电路部分的原理图如图 3.4 所示， 由于 FPGA 板卡的 J5 部分接口只支持 1.8V电平，不能使用 5V 电平直接作为上拉电平，因此使用线性稳压器 RT9013-18GB 将 5V 电压降压到 1.8V。

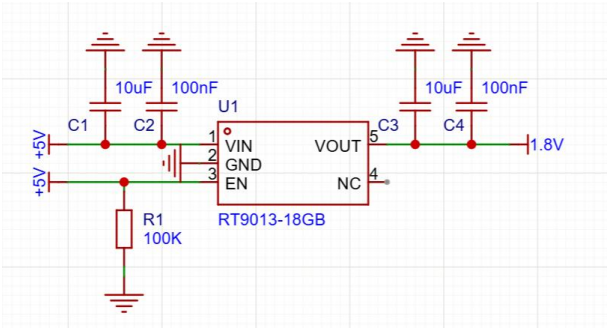


图 3.4 键盘电源部分原理图

（3）按键电路实现

自制键盘的按键电路实现如图 3.5 所示，每路 GPIO 通过上拉电阻连接至 1.8V，按键闭合时将输入接地。因此，松开时输入为高电平，按下时为低电平。

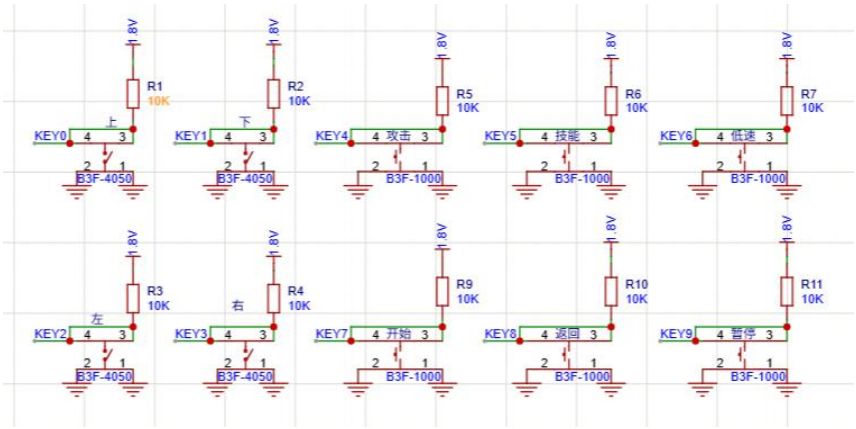


图 3.5 键盘电路实现原理图

（4）底板电路 PCB 的正反面

自制键盘电路 PCB 的正反面分别如图 3.6、图 3.7 所示。

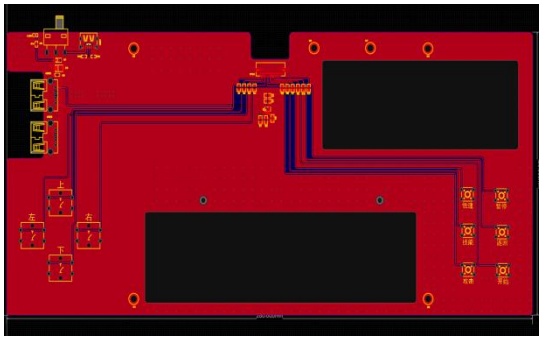


图 3.6 键盘电路 PCB 正面

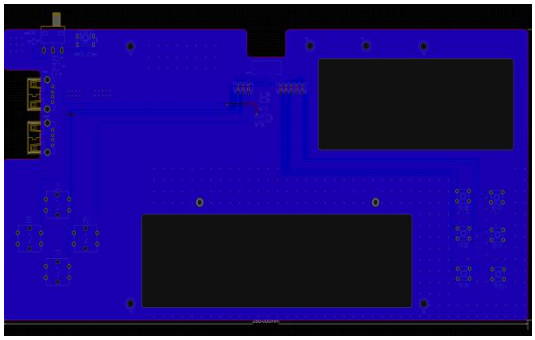


图 3.7 键盘电路 PCB 反面

3.3 软件系统介绍

① 软件整体介绍

软件整体流程图如图 3.8 所示。

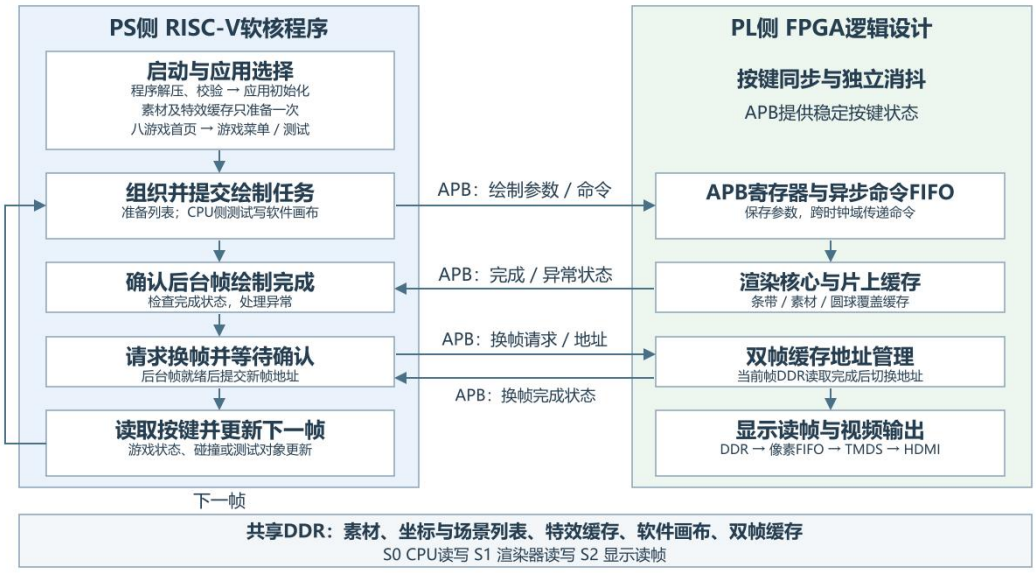


图 3.8 软件整体流程图

本工程的软件由 RISC-V 软核程序和 FPGA 逻辑两部分组成。软核程序以 C 语言为主，启动搬运和栈切换使用 RISC-V 汇编，负责程序与素材初始化、首页选择、游戏逻辑及绘制任务组织。FPGA 逻辑采用 Verilog 编写，负责命令执行、像素处理和视频输出。两部分通过 APB 寄存器交换命令与状态，通过 DDR 共享素材、坐标列表和帧缓存。

固化版本上电后先解压并校验应用程序，再装载各游戏素材，完成后进入八游戏首页。各游戏共用渲染驱动，按自身规则处理输入和更新画面；性能测试入口位于东方红魔乡的模式菜单。普通绘制由软核提交命令，确认硬件完成后请求换帧。同屏对比的 CPU 侧分片生成独立软件画布，完成后复制到显示区域，与 FPGA 侧同时展示。PL 侧的按键、渲染和显示通路由寄存器及共享存储器连接。

② 软件各模块介绍

（1）PL 端设计

PL 端设计围绕命令接收、像素处理和帧缓存显示展开。以下介绍本工程自行实现的主要模块；总线信号在接口图中按通道分组，输入、输出方向均以模块为参照。

① 按键同步与消抖模块（keyboard.v）

下图 3.9 为按键同步与消抖模块的接口示意图。



图 3.9 独立按键同步与消抖模块接口示意图

接口方面，i_clk 为按键采样时钟，i_rstn 为低有效复位，i_key[9:0]连接自制键盘的十路独立 GPIO。o_key_state[9:0]输出确认后的按键状态，并送至 APB 按键寄存器；按下对应位为 1，松开为 0，长按期间状态持续保持。

内部先通过两级寄存器同步外部输入，再为每个按键分别设置稳定计数器。输入与已确认状态不一致并连续达到消抖门限后才更新输出，短暂抖动则清零计数。按下与松开均进行消抖，各键互不影响；工程按 100MHz 采样时钟配置 20ms 门限。

② APB 渲染控制寄存器模块（apb_render_regs.v）

下图 3.10 为 APB 渲染控制寄存器模块的接口示意图。

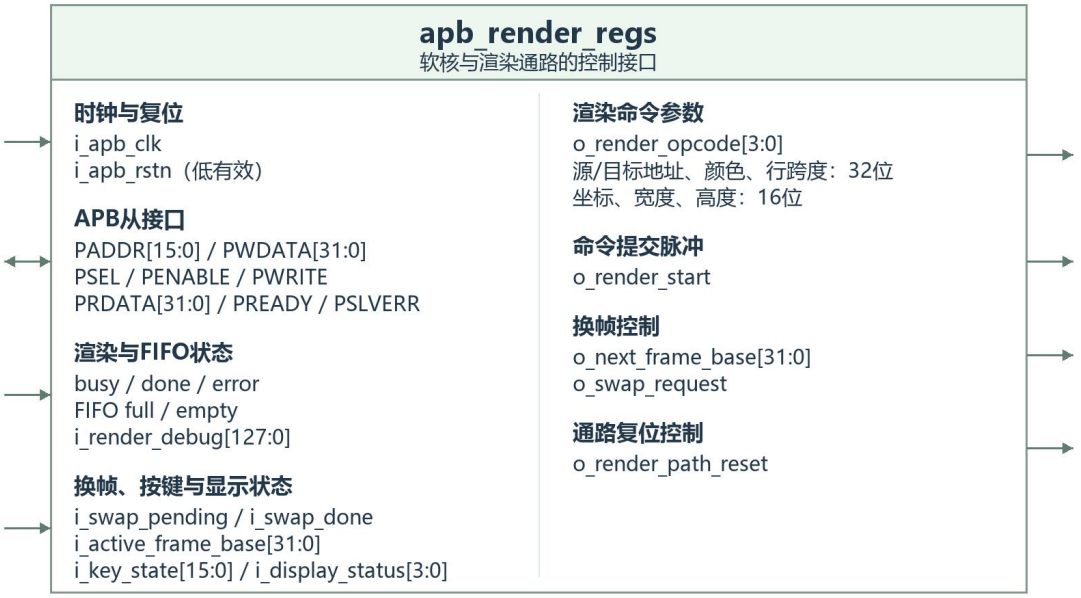


图 3.10 APB 渲染控制寄存器模块接口示意图

APB 接口接收软核对地址、数据和控制寄存器的访问，返回读取数据及访问完成信号。命令输出组包括操作码、素材地址、目标帧地址、坐标、宽高、颜色及源/目标行跨度；o_render_start 用于提交一次命令。状态输入组汇集渲染器、命令 FIFO、换帧管理器、按键和显示端的反馈，供软核统一读取。

内部按照寄存器地址译码，先保存绘制参数，再由 control 写操作产生启动脉冲；顶层将参数打包写入异步命令 FIFO，送往渲染时钟域。换帧地址和请求分别输出，path_reset 提供渲染/显示通路复位控制。该模块只交换参数与状态，不搬运素材像素。

③ 二维硬件渲染核心模块（render_core.v）

下图 3.11 为二维硬件渲染核心模块的接口示意图。



图 3.11 二维硬件渲染核心模块接口示意图

命令接口以 i_cmd_valid 与 o_cmd_ready 同时有效作为接收条件，锁存本条命令的操作码和绘制参数。AXI 主接口通过 AR/R 通道读取素材、坐标列表及混合所需的目标像素，通过 AW/W/B 通道写回帧缓存，数据宽度为 128 位。o_busy 表示执行中，o_done 和 o_error 分别报告完成与错误；四组 32 位调试输出保留首次异常的原因及地址。

内部状态机将地址计算、突发配置、读数据、像素处理和写响应分步执行。五类命令分别实现矩形填充、图像搬运、批量精灵、坐标生成圆球弹幕和可调全局 Alpha 的条带场景。图像路径支持 Color Key、水平镜像及可调全局 Alpha；满足对齐且无镜像、无 Alpha 的搬运采用最多 16 拍的缓冲式突发复制。批量命令从 DDR 列表读取实例坐标，圆球弹幕直接生成彩色外环与白色圆心，减少命令提交及素材读取。

RGB565 混合分别处理 R、G、B 分量，采用/256 的移位实现，并对完全透明和不透明端点单独处理。写入使用字节选通保护矩形边缘，突发长度受行尾与 4KB 边界限制；工程同时启用目标帧范围及批量素材、坐标表范围检查，避免非法命令写坏其他存储区。

批量圆球命令还可调用内部子模块 bullet_coverage_buffer.v。模块从 DDR 读取坐标表，将绘制区域按八个像素 Bank 组织，每像素用 2 位编码表示未覆盖、彩色外环或白色圆心。按列表顺序处理后，缓存保留该批次的最终覆盖结果，再按行组成 128 位数据并突发写回；未覆盖像素通过写选通保留原背景。该通路不读取圆球贴图，也不将透明混合简化为覆盖。

当帧地址对齐、目标行跨度为 1280 字节等条件满足时，8×8 和 16×16 圆球在单条命令达到 1024 颗后使用覆盖缓存，32×32 圆球使用该通路；小批次仍可使用原有绘制路径。该子模块的 AXI 读写通道由 render_core 内部复用，不改变外部命令格式及三路 AXI 结构。

④ Alpha 条带缓存与并行混合模块（alpha_strip_buffer.v）

下图 3.12 为 Alpha 条带缓存与并行混合模块的接口示意图。

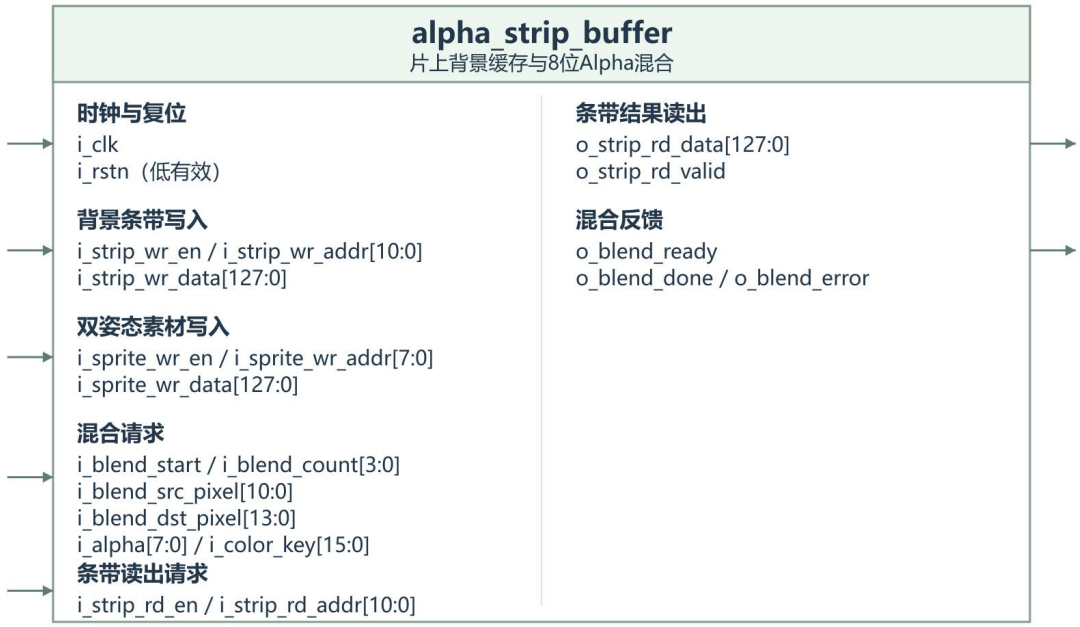


图 3.12 Alpha 条带缓存与并行混合模块接口示意图

条带写入和素材写入接口由 render_core 提供 128 位数据，读出接口返回混合后的条带及有效标志。混合请求给出源/目标起始像素序号、1～8 个像素的处理数量、透明色值及 8 位 Alpha 参数 i_alpha[7:0]。o_blend_ready、o_blend_done 和 o_blend_error 用于协调请求接收、处理完成及异常反馈。像素地址以 RGB565 像素为单位，缓存读写地址以 128 位数据拍为单位，模块本身不直接连接 DDR。

默认缓存 640×16 像素的背景条带，共 20KiB，并缓存两张 30×30 素材。背景与素材均按像素序号低三位拆成八个 16 位存储 Bank，支持非八像素对齐的连续访问。混合单元采用读取、差值计算、乘法和结果合并等流水阶段，并行处理 RGB565 像素。Alpha 为 128 时使用加法与右移，其余中间值采用整数加权计算；0 和 255 分别按保持背景和直接复制处理，透明色像素不修改背景。同一 Bank 存在尚未完成的同地址读写依赖时，暂停接收相关请求，保证连续叠加结果正确。

软核将精灵按其覆盖的条带组织描述符，render_core 逐条带装入背景，在片上完成该区域所有精灵的叠加，再集中写回 DDR。这样背景不必为每个重叠精灵反复从 DDR 读取和写入，收益主要来自访存次数的减少，而不是只加快一次混合运算。

⑤ 双帧缓存地址管理模块（frame_buffer_manager.v）

下图 3.13 为双帧缓存地址管理模块的接口示意图。

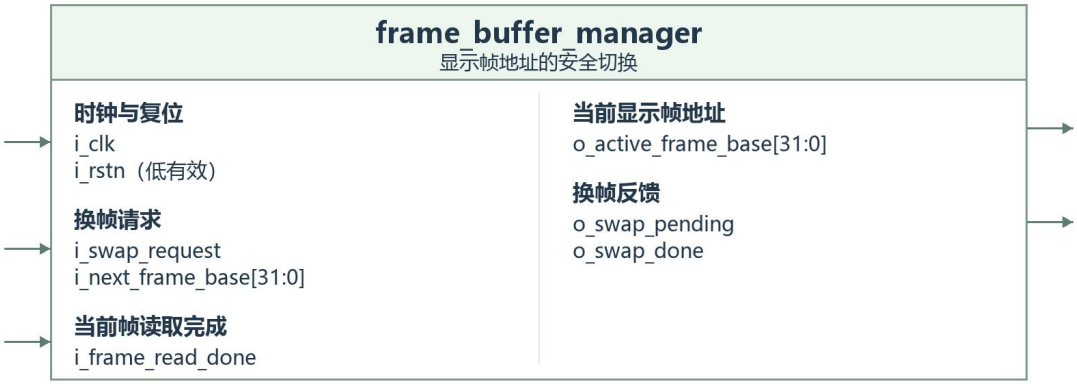


图 3.13 双帧缓存地址管理模块接口示意图

i_next_frame_base 与 i_swap_request 接收已跨域的下一帧地址和换帧请求，i_frame_read_done 连接帧读取器的 o_done。o_active_frame_base 直接提供给帧读取器，pending 表示已有待处理请求，done 为换帧完成脉冲，状态同步回 APB 后由软核确认。

模块只管理地址，不搬运整帧数据。请求到来后先保存候选地址，待当前帧 DDR 读取全部结束才更新活动帧地址，使下一次读取使用新帧；尚有请求等待时不接收第二个请求。软核只向后台帧绘制，确认渲染完成后申请切换，并在换帧完成后复用旧帧。这里的切换条件是 DDR 读帧完成，不是直接检测 HDMI 场同步沿。

⑥ DDR 帧读取与像素输出模块（frame_reader_axi.v）

下图 3.14 为 DDR 帧读取与像素输出模块的接口示意图。

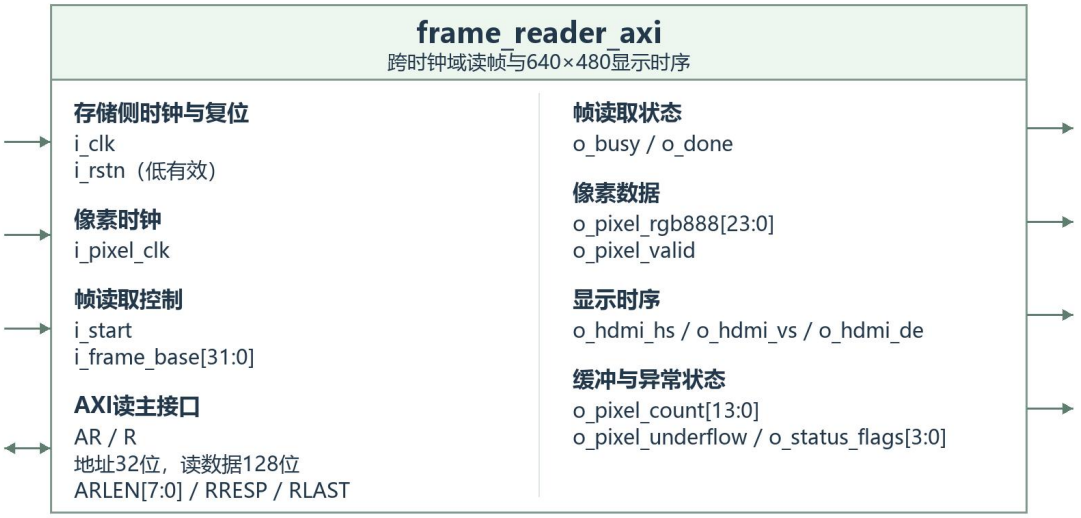


图 3.14 DDR 帧读取与像素输出模块接口示意图

存储侧通过 i_clk 执行 AXI 读取，像素侧通过 i_pixel_clk 输出像素。i_start 控制读帧，i_frame_base 指定帧首地址；AXI 接口仅使用 AR/R 两个通道。RGB888、HS、VS 与 DE 交给后级视频编码接口，o_done 反馈给帧地址管理器。像素计数、下溢及状态标志提供给诊断通路；o_done 表示 DDR 读帧完成，不等同于整帧已经显示完毕。

内部按行生成突发读请求，限制单次长度不越过行尾和 4KB 边界，并在请求前为整次突发预留 FIFO 空间。异步像素 FIFO 完成 128 位到 16 位的宽度转换及存储时钟到像素时钟的跨域，像素端将 RGB565 扩展为 RGB888 并生成 640×480 行场时序。当前集成预存 4096 个像素后才在帧首启动输出，以缓冲 DDR 服务延迟。

模块检查读响应、RLAST 拍数及 FIFO 异常；读响应错误以黑色数据替代对应数据拍，避免后续像素整体移位。发生显示下溢后记录故障并停止继续读取像素，恢复时需先安全结束 AXI 事务再复位通路，不在帧中途直接续播，防止错行与彩屏。视频编码和物理引脚输出沿用已有接口，不属于本节模块的内部逻辑。

上述模块在顶层完成连接：APB 侧命令经异步 FIFO 进入 render_core，渲染核心通过 AXI 写入后台帧；frame_buffer_manager 控制前帧地址，frame_reader_axi 持续读取前帧。顶层另实现换帧请求和完成事件的跨域传递、首次错误快照同步，以及通路复位在存储和像素时钟域的异步置位、同步释放。按键状态经寄存器回到软核，形成输入、逻辑更新、绘制和显示的完整闭环。

（2）PS 端设计

PS 端负责游戏运行与绘制任务组织，程序按以下六部分介绍。八款游戏各自维护状态，图像输出共用渲染驱动和双帧缓存，不需要为每个游戏重新编写硬件接口。

① 程序启动与素材装载

为满足启动 ROM 的加载容量，固化程序由解压引导程序和压缩应用组成。boot 目录中的代码负责解压及校验，再由片上 RAM 中的搬运代码将应用送至运行地址并跳转执行。解压失败时先重试，仍失败则输出错误并停止进入应用，避免执行不完整的程序。

进入 game_control.c 后，程序先确认 PL 通路就绪并显示 Loading，再暂时复位渲染与显示通路，完成素材搬运和解压。ddr_init.c 装载东方原始素材，校验失败后重新搬运；handheld_assets.c 装载首页及新增游戏的压缩资源包，每包进行 CRC 校验，最多尝试三次，失败后禁用对应游戏入口，其他已就绪的游戏仍可使用。素材装载完成后，在原有通路复位期间生成特效图案和受击轮廓缓存，再恢复显示并进入 Ready 首页，返回首页或切换游戏时不再重复加载或生成。

② 首页与应用切换

handheld.c 负责统一首页，八个游戏图标按四列两行排列。方向键切换选中项，KEY7 进入，游戏菜单中的 KEY8 返回首页；游戏运行中的返回操作由各自界面处理。game_control.c 根据选择调用对应入口，游戏退出后重新显示首页。

东方红魔乡依次提供普通游戏、无敌游戏、CPU 与 FPGA 同屏对比、Alpha 精灵测试，以及 8×8、16×16、32×32 三种 GPU 弹幕测试，共七个模式。普通和无敌模式进入选人界面，左右键选择灵梦或魔理沙，上下键选择 EASY、NORMAL、HARD 三档难度。难度选项在两位角色之间纵向排列，当前项高亮。

③ 渲染驱动与帧调度

ctrl_render.c 统一封装填充、图像搬运、镜像、Alpha 及批量绘制接口，render_regs.h 规定寄存器地址和命令格式。驱动检查参数范围、等待 FIFO 空间，再写入参数并触发命令；坐标表和场景表提交前执行内存同步。东方关卡每帧按提交绘制、等待完成、请求换帧、更新逻辑的顺序运行，PL 读取列表时不修改其内容。棋盘和菜单等静态界面仅在状态变化时重绘，减少无效提交。

④ 游戏逻辑与界面绘制

东方红魔乡由玩家、武器和关卡模块分工实现。player.c 处理移动、边界及低速切换，主副武器模块管理发射、命中和绘制。game_stage.c 统一调用两关接口，stage1.c 和 stage2.c 维护小怪、Boss、敌弹及道具。大量敌弹使用定点坐标更新，同类对象按素材或颜色归组绘制。两关之间保留分数和资源，死亡或通关后显示所选角色与最终得分。

game_start.c 处理东方的菜单、选人和渐隐切换，game_view.c 绘制滚动背景与右侧信息面板。面板内容未改变时复用已绘制结果，游戏区域每帧重新覆盖。灵梦技能吸收并消除敌弹，魔理沙技能在 300 个游戏帧内提高至三倍伤害；两者均显示对应立绘。

其余七款游戏分别保存自身状态。象棋实现本地双人走子、合法性判断和悔棋；扫雷处理开格、标记及胜负；2048、贪吃蛇、推箱子和俄罗斯方块分别处理合并、移动、推箱与落块规则。地牢幸存者管理自动攻击、敌人生成、经验升级和 Boss 战。规则计算与界面绘制分开，图像、文字和动画仍由同一套渲染接口提交。

game_fx.c 为东方和幸存者提供共用特效，包括受击轮廓闪光、击破粒子和武器拖尾，幸存者升级时显示扩散光环。轮廓由已有素材生成并保留透明区，粒子与光环由程序生成，利用 Color Key 和硬件 Alpha 叠加、渐隐，不新增 Flash 素材包。特效池最多容纳 64 个粒子，每帧最多绘制 8 处受击轮廓；超出容量时跳过额外装饰，不改变玩法。特效随游戏帧更新，暂停时停止推进。

⑤ 渲染对比与压力测试

split_benchmark.c 实现 CPU 与 FPGA 同屏对比，左侧由 FPGA 绘制，右侧由 CPU 生成软件画布，初始各 10000 颗 8×8 弹幕，上下键每次增减 500 颗。两侧使用相同大小的绘制区域、圆球掩码、颜色及基于时间的运动规则。CPU 将清屏和绘制分成短任务，在硬件执行及换帧等待期间逐步推进；完成整张画布后才发布，未完成时继续显示上一张完整结果。两侧分别统计实际更新帧率和 DRAW MS，CPU 侧的统计不包含结果呈现。

render_benchmark.c 提供三种独立 GPU 弹幕测试，8×8、16×16、32×32 的初始数量分别为 40000、20000、7000 颗，上下键每次增减 500 颗，范围为 500 ~ 48000 颗。benchmark_motion.c 预先生成运动数据，运行时用按尺寸编译的整数内核更新打包坐标，降低软核的坐标准备开销。界面显示绘制耗时、数量和实际 FPS，串口分段统计坐标准备、命令提交、完成等待、统计绘制及换帧时间。

Alpha 测试初始使用 5000 个 30×30 双姿态精灵，Alpha 为 128/256，并开启透明色剔除。精灵始终在屏幕内运动并碰壁反弹，上下键按 100 只调节数量，范围 100 ~ 10000 只；

左右键按 16/256 调节 Alpha，与数量控制相互独立。界面显示数量、透明度及 FPS，掉帧时不自动减少精灵。

⑥ 异常诊断与存储保护

frame_perf.c 分别统计命令提交、渲染完成等待、换帧等待和逻辑计算时间。cpu_diag 与 cpu_trap.S 保存执行进度和异常现场，渲染驱动记录首次总线错误及访问地址。普通帧等待超时或出现显示故障时，不提交未完成的画面，而是恢复通路后返回菜单；Alpha 测试在恢复后重试，并保留 KEY8 返回操作。串口只按统计窗口或在错误、恢复时输出，避免日志影响帧率。

东方关卡、CPU 与 FPGA 同屏对比、GPU 弹幕测试、Alpha 测试、幸存者和推箱子 / 俄罗斯方块入口使用片上 RAM 栈，退出后恢复原栈。它们按顺序运行，复用同一片上栈区，并检查保护标记。handheld_memory_map.h 对常驻素材、解压暂存区、列表和生成缓存的边界加入编译期检查，防止地址重叠。数字字库和背景缓存也独立分区，避免后加载的素材覆盖先生成的内容。

四、完成情况及性能参数

4.1 整体介绍

整个系统的实物图如图 4.1 所示，作品以掌机形式实现，完成了基于易灵思 Ti60F225 FPGA 的二维图像渲染加速系统，由 FPGA 开发板、HDMI 显示屏及自制掌机底板组成。系统采用 RISC-V 软核处理游戏逻辑和绘制任务，专用硬件渲染器完成像素搬运与混合运算，输出 640×480 分辨率、约 59.52Hz 的图像。硬件配置、压缩软核程序及八款游戏素材可合并为完整 Flash 镜像，固化后可独立启动，无需连接计算机下载程序。系统提供同屏渲染对比、三种尺寸的弹幕测试和可调 Alpha 精灵测试，分别用于观察绘制效率、对象承载量与透明叠加性能。



图 4.1 完整作品实物图

4.2 性能测试展示

图 4.2 为 CPU 与 FPGA 同屏渲染对比画面。两侧各绘制 10000 颗 8×8 圆球，分别显示实际更新帧率和绘制耗时。

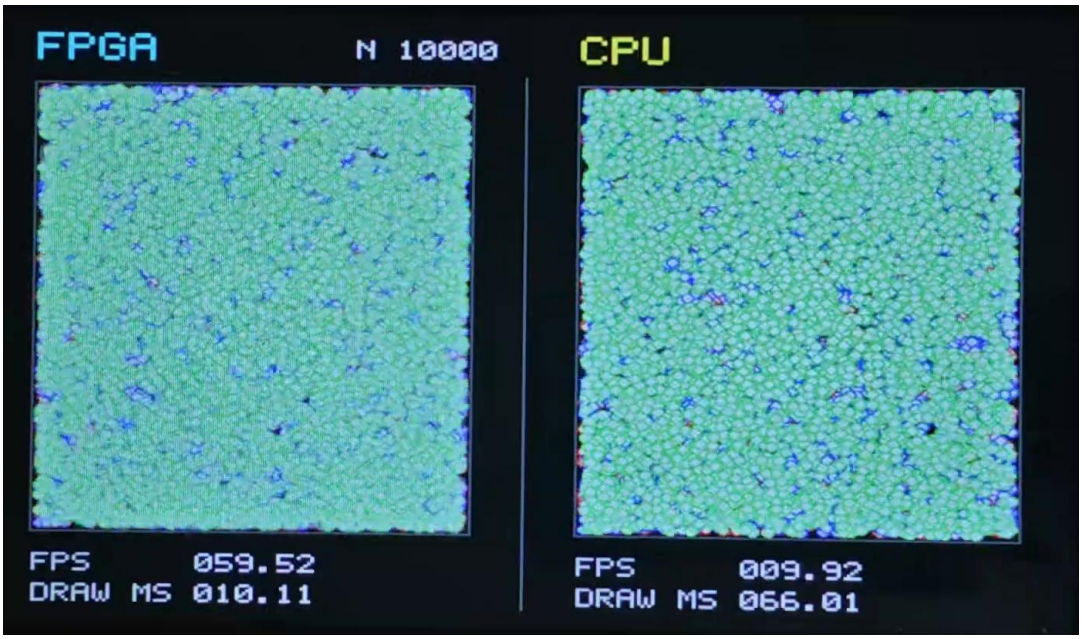


图 4.2 CPU 与 FPGA 同屏渲染对比

图 4.3 为 Alpha 混合与透明色剔除同时开启的测试画面，使用 5000 个 30×30 双姿态精灵，Alpha 设为 128/256。

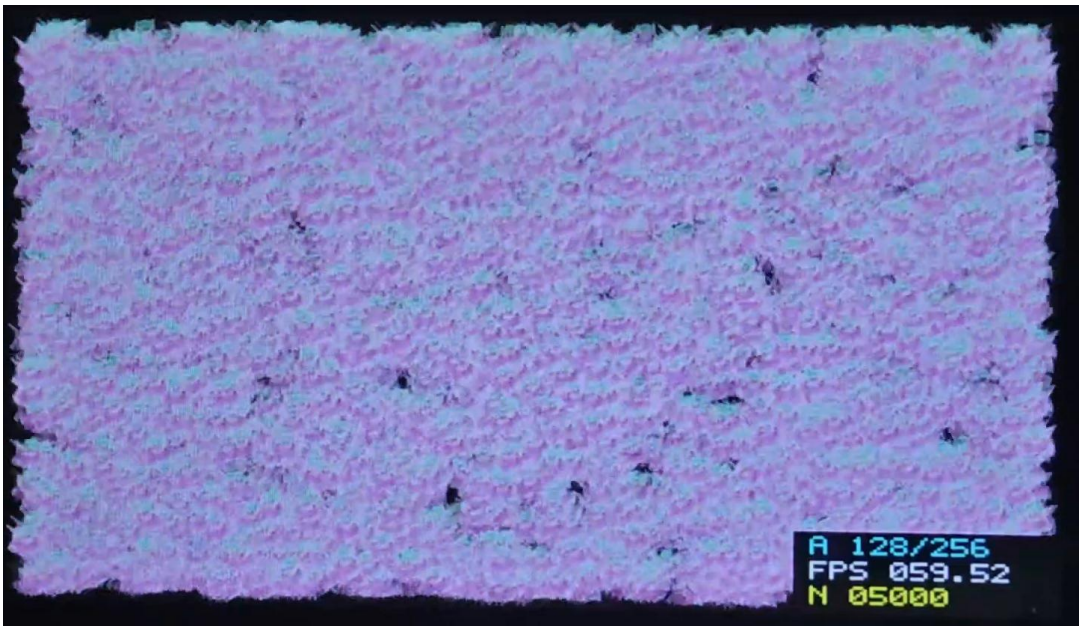


图 4.3 Alpha 混合与透明色剔除测试

图 4.4 依次展示 8×8、16×16 和 32×32 弹幕测试，当前数量分别为 40000、20000 和 7000 颗。各子图下方为同一帧统计区域的局部放大，截图反映对应时刻的读数，汇总结果见表 2.2。

(a) 8×8 · 40000颗　(b) 16×16 · 20000颗　(c) 32×32 · 7000颗

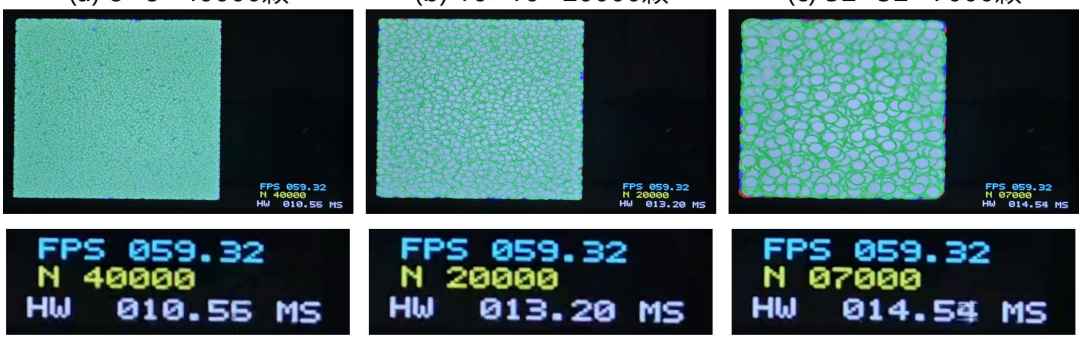
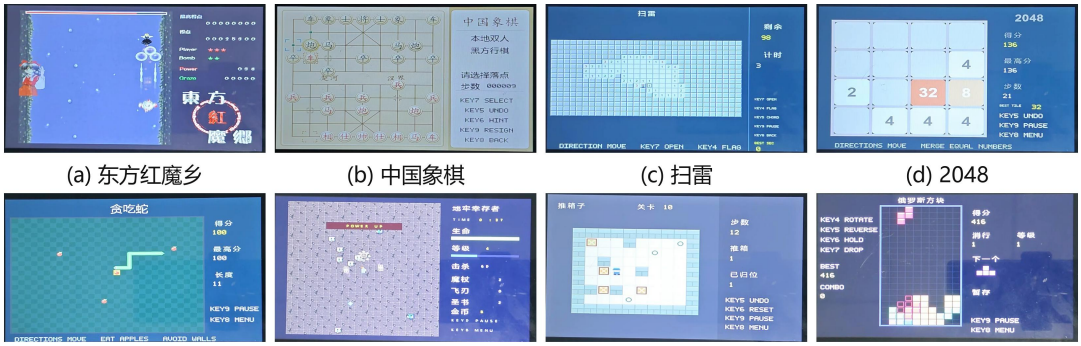


图 4.4 三种尺寸的 GPU 弹幕压力测试

4.3 游戏功能展示

图 4.5 展示八款游戏的实际运行画面，按首页顺序排列为两行四列。各游戏均通过掌机按键操作，并共用硬件渲染与显示通路。



(a) 东方红魔乡　(b) 中国象棋　(c) 扫雷　(d) 2048

(e) 贪吃蛇　(f) 地牢幸存者　(g) 推箱子　(h) 俄罗斯方块

图 4.5 八款游戏运行画面

五、总结

5.1 可扩展之处

目前系统已实现八款游戏和多种渲染测试，后续可从以下方面扩展：

① 完善渲染功能。在现有填充、搬运、镜像和 Alpha 混合的基础上，增加硬件图像缩放、旋转与可配置裁剪窗口，减少 CPU 参与图像变换的开销。

② 扩大缓存适用范围。当前条带加速通路主要面向两张 30×30 素材，可进一步支持不同尺寸和更多种素材，提高复杂场景中的缓存利用率。

③ 完善应用平台。沿用统一首页和渲染接口加入更多游戏，将已有的最高分、通关进度等持久化记录扩展为完整对局存档，并增加音效及设置功能。

④ 完善掌机结构。完成外壳、屏幕与底板的固定设计，增加电池供电和电量检测，提高便携性。

5.2 心得体会

刚开始做这个项目时，我们想的是先显示画面，再把游戏跑起来，性能不够就继续加速。做下去才发现，能显示、能运行和能稳定使用，需要分别验证。从移动色块到八个小游戏，每增加一项功能，都可能暴露出之前没考虑的问题。这个过程最大的锻炼，是逐渐学会了怎么判断问题、验证想法。

印象比较深的是性能优化。看到弹幕多了掉帧，我们最初想到的是加大 FIFO、提高主频，觉得硬件跑得更快，问题应该就能解决。后来把一帧拆成命令提交、渲染完成等待、换帧等待和逻辑计算，才发现不同阶段的瓶颈并不一样。大量对象逐个下发命令时，要先减少提交次数；Alpha 混合反复访问 DDR 时，则需要考虑数据能不能留在片上重复使用。批量指令、坐标生成弹幕和条带缓存的效果让我们认识到，优化往往不是让原来的操作做得更快，而是减少没有必要的操作。后期大量弹幕重叠时，我们先在片上汇总覆盖结果，再统一写回，并预先准备运动数据，减少每帧坐标计算。时序报告通过，也不能代替上板验证。

花屏和卡死则让我们改掉了不少凭感觉判断的习惯。出现横条、底部彩块和角色隔行显示时，我们曾怀疑 DDR 里的数据被改坏了，但相似的画面异常，可能涉及越界访问、显示下溢、启动状态或素材地址重叠。尤其后来幸存者数字显示异常，最终查到生成的字符缓存与新增素材区域冲突，让我们意识到，地址规划必须随着功能一起维护，不能只看某个模块内部是否正确。

卡死问题更考验耐心。我们一度觉得是程序太长，甚至怀疑软核的 PC 不可靠。后来通过异常记录、执行进度和保留现场，才逐步把注意力放到具体的异常地址、栈空间和存储布局上。相比反复重新烧录，留下第一次出错的现场更有价值。我们也开始习惯一次只改一个关键因素，先确认修改前后的差别，而不是把几个猜测一起改进去，最后连为什么恢复正常都说不清。

启动和固化也给我们上了一课。调试下载能运行，不代表断电后还能运行。PL、PS 的复位顺序、素材搬运校验，以及完整固化文件是否包含正确的程序和素材，都需要单独确认。最后通过完整镜像烧录、回读校验和断电重启测试，才算真正把开发环境里的工程变成了可以独立使用的作品。

回头看，这个项目让我们形成了更踏实的习惯：先打通最小通路，再逐步加功能；修改前备份，出错时留住现场；用仿真检查协议和边界，用上板测试确认实际效果。现在遇到问题，我们仍会先猜原因，但不会急着把猜测当成结论。把偶然跑通变成可以解释、复现和交付的结果，是这次开发最大的收获。